



INFORMATION SECURITY LAB MANUAL

Created By

Hashim Yaseen Mulug-1501

**School of Information Technology
MUL, Lahore**

Contents

INTRODUCTION TO INFORMATION SECURITY:	4
INFORMATION SECURITY.....	4
Real word application.....	4
Need for Information security.....	4
Wireshark:	5
GETTING STARTED WIRESHARK:	5
USER INTERFACE	5
START SCREEN:	5
TOP BAR:	7
MENU BAR	10
MAIN SCREEN.....	18
FILTERS:	19
BLUETOOTH.....	19
HTTP	20
STP.....	20
UDP.....	21
TCP	21
IP	22
CONVERSATIONS:.....	22
Location	22
Conversations Screen.....	23
Types of Conversations	23
Protocols filter.....	23
Conversation properties	24
VOIP CALLS	25
DOWNLOAD AND IMPORT TO WIRESHARK.....	25
PLAY STREAMS	25
I/O GRAPHS:	28
LOCATION	28
GRAPHS SCREEN:	29
GRAPH INFORMATION	29
GRAPH CONTROLS	29
Encryption and Decryption	30

Caesar Cipher (Shift Cipher):30

 Explanation:30

 How it works30

 Code:30

Symmetric Encryption (Secret Key)31

Asymmetric Encryption (Public Key).....32

 The Concept:32

Monoalphabetic Cipher33

One-Time Pad (OTP)33

Transposition Cipher34

INTRODUCTION TO INFORMATION SECURITY:

Information Security (**InfoSec**) is the practice of protecting information and the systems that handle it. It isn't just about stopping hackers; it's about ensuring that data whether digital or physical remains safe, accurate, and available to those who need it.

INFORMATION SECURITY.

At its core, InfoSec is defined by the **CIA Triad**, a model designed to guide security policies:

Confidentiality: Ensuring only authorized people can access sensitive data (e.g., using passwords or encryption).

Integrity: Guaranteeing that data has not been tampered with or altered (e.g., digital signatures).

Availability: Ensuring that systems and data are ready for use when needed (e.g., protecting against system crashes or DDoS attacks)

Real word application

InfoSec is active in almost every interaction you have with technology:

Online Banking: Encrypting your session so a bystander at a coffee shop can't see your account balance.

Healthcare: Protecting patient records (Electronic Health Records) to comply with privacy laws like HIPAA.

E-commerce: Securing credit card details during a checkout process.

Social media: Multi-factor authentication (MFA) to prevent unauthorized logins.

Need for Information security

Why do we invest so much in this field?

Preventing Financial Loss: Cyberattacks cost companies billions in stolen funds and recovery expenses.

Protecting Reputation: A single data breach can destroy customer trust overnight.

Legal Compliance: Governments mandate that companies protect user data (e.g., GDPR or CCPA). Failure to do so leads to massive fines.

National Security: Protecting critical infrastructure like power grids, water systems, and military communications.

Wireshark:

Wireshark is the world's most widely used **Network Protocol Analyzer**. Think of it as a "microscope for the network." It allows you to see exactly what is happening inside your network cables at a microscopic level.

GETTING STARTED WIRESHARK:

USER INTERFACE

The Wireshark interface is where the "magic" happens. It might look intimidating at first, but it is very logical.

START SCREEN:

This is your home base. It lists all available network interfaces (WiFi, Ethernet, etc.). You'll see a "sparkline" (a small moving graph) next to the interface that is currently active.

Action: Double-click the active interface to start "sniffing" traffic.

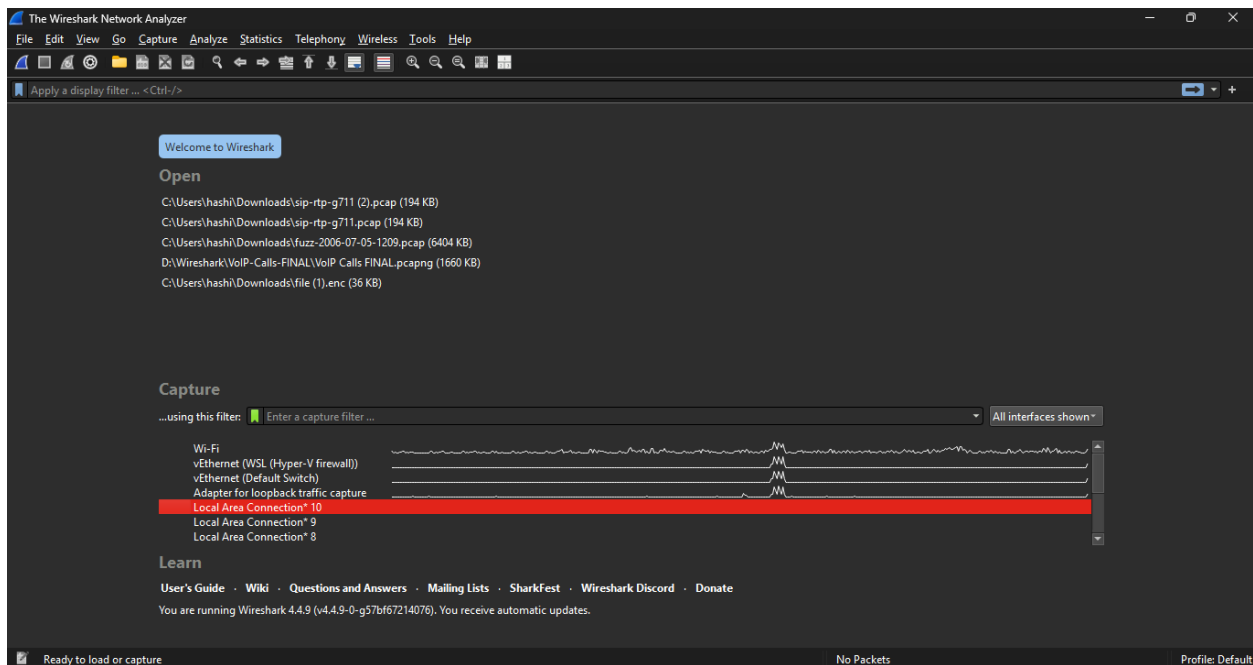


Figure 1

FILTER SECTION:

Located just below the toolbar, this bar turns **Green** if your syntax is correct and **Red** if it's wrong.



Figure 2

OPEN RECENT FILES

As the name suggests, this feature maintains a history of the packet capture files (.pcap, .pcapng, or .cap) that you have recently opened or saved.

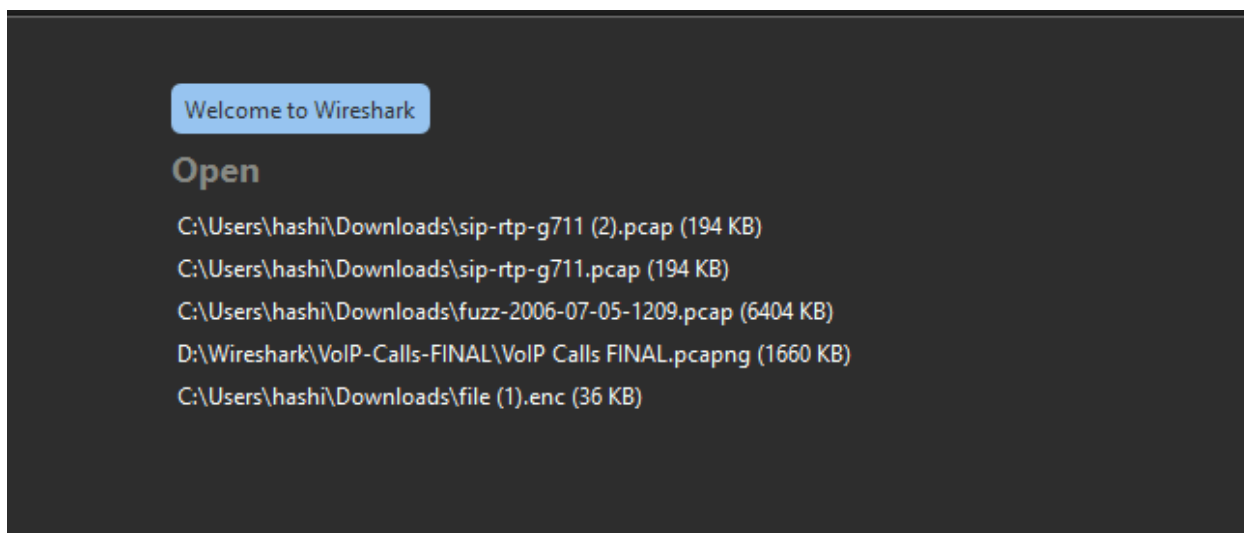
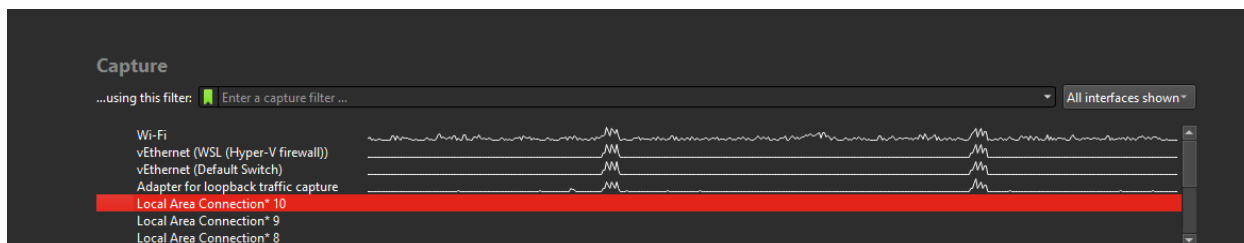


Figure 3

ACTIVE NETWORK CONNECTIONS

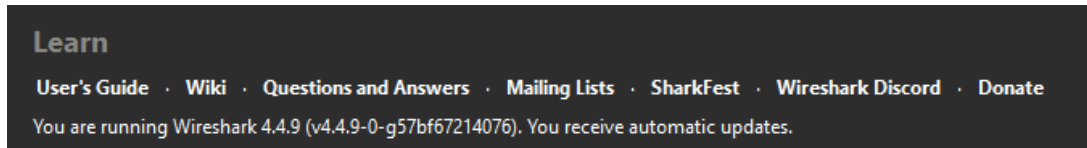
While the main screen shows you every individual packet, the "Active Connections" or **Endpoints/Conversations** windows summarize that data into a list of "who is currently active." It identifies every device that has sent or received at least one packet during the capture.





FROM WIRESHARK COMMUNITY

Wireshark is **Open Source** software. This means it isn't owned by a single company that keeps its secrets locked away. Instead, it is maintained by a global community of experts. This community ensures that Wireshark stays free, transparent, and up-to-date with the latest internet protocols.



TOP BAR:

The Top Bar consists of a row of icons located directly below the Menu Bar. While there are many icons, they are logically grouped into four main "Zones."



Figure 4

START CAPTURE

Begins recording network traffic on the selected interface.



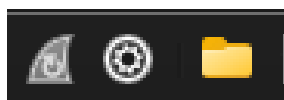
STOP CAPTURE

Stops the recording. Use this before analyzing data to keep the packet list from jumping.



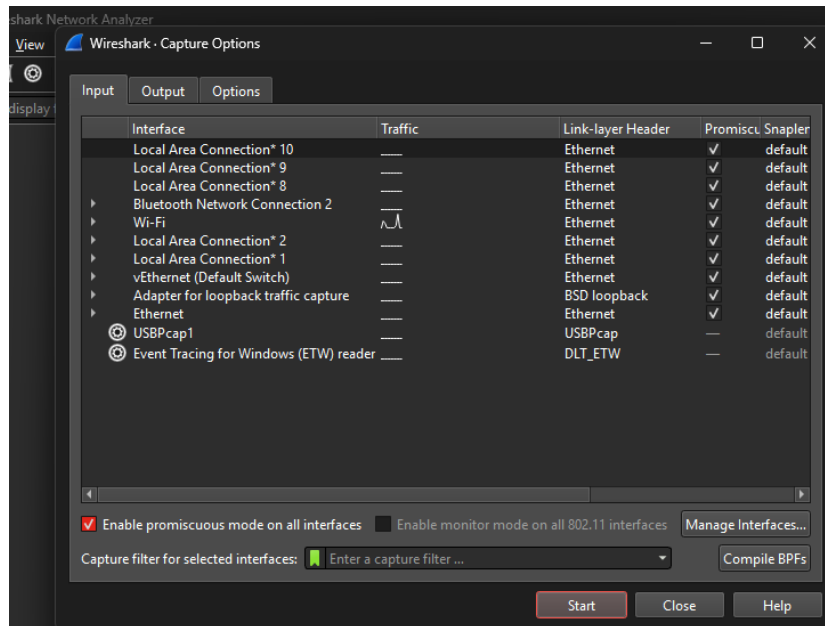
RESTART CAPTURE

: Quickly wipes the current data and starts a fresh capture.



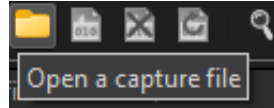
CAPTURE OPTIONS

Allows you to choose which network card (Wi-Fi, Ethernet) to use and set specific capture rules.



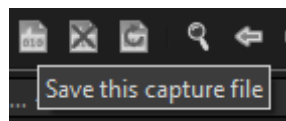
OPEN CAPTURE

Opens a previously saved .pcap file.



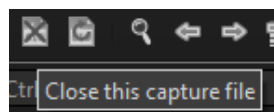
SAVE CAPTURE

Saves your current packets for future evidence or analysis.



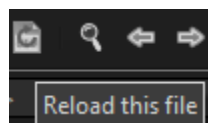
CLOSE CAPTURE

Closes the current file to return to the Start Screen.



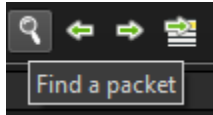
RELOAD CAPTURE

Refreshes the current file (useful if the file was modified by another tool).



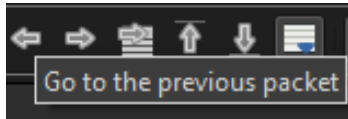
FIND PACKET

Search for specific text (like a password) or hex values within the packets.



PREVIOUS CAPTURE

Move back through your history of selected packets.



NEXT CAPTURE

Move forth through your history of selected packets.



SPECIFIC CAPTURE

Jump directly to a packet by its number.



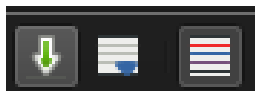
FIRST CAPTURE

Instantly jump to the very beginning or the very end of the capture.



LAST CAPTURE

Instantly jump to the very beginning or the very end of the capture.



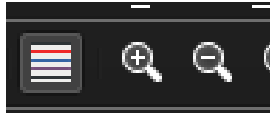
AUTO SCROLL

When enabled, the screen follows the newest packets in real-time.



DRAW PACKET

Toggles the coloring rules (like Green for HTTP) on and off.



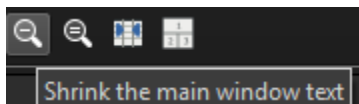
ENLARGE WINDOW

Adjusts the font size for better visibility.



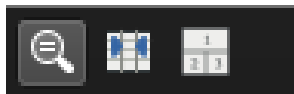
SHRINKING WINDOW

Adjusts the font size for better visibility.



REVERT TO NORMAL

Adjusts the font size for better visibility.



RESIZE LIST

Automatically fits the column widths to the text inside them.

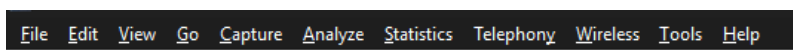


RESET LAYOUT

Returns all windows to the default Wireshark view.

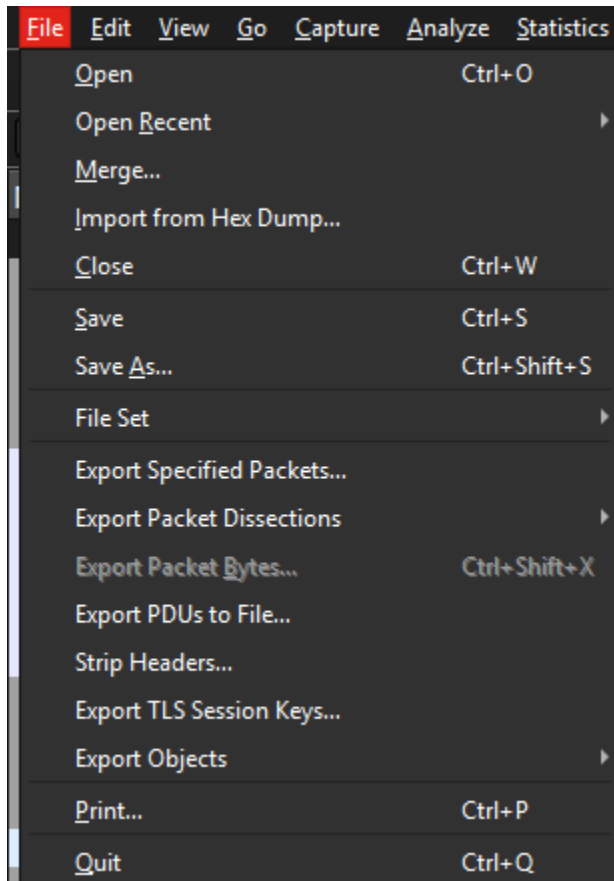


MENU BAR



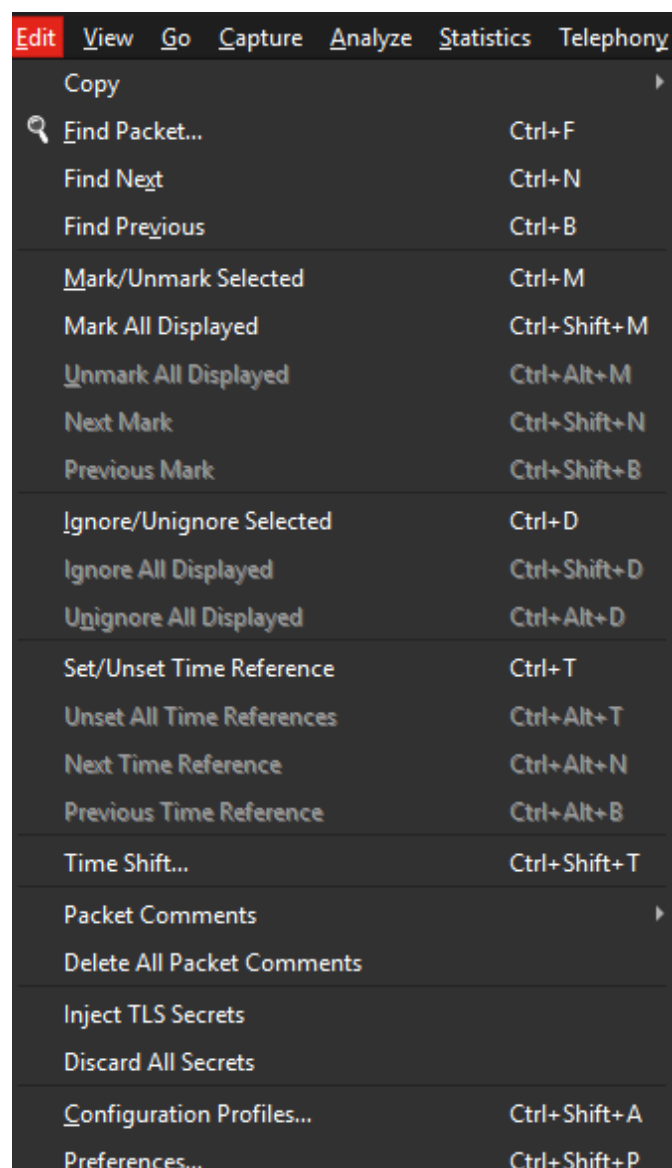
FILE MENU

Core operations like Open, Save, Export (for specific packets), and Print.



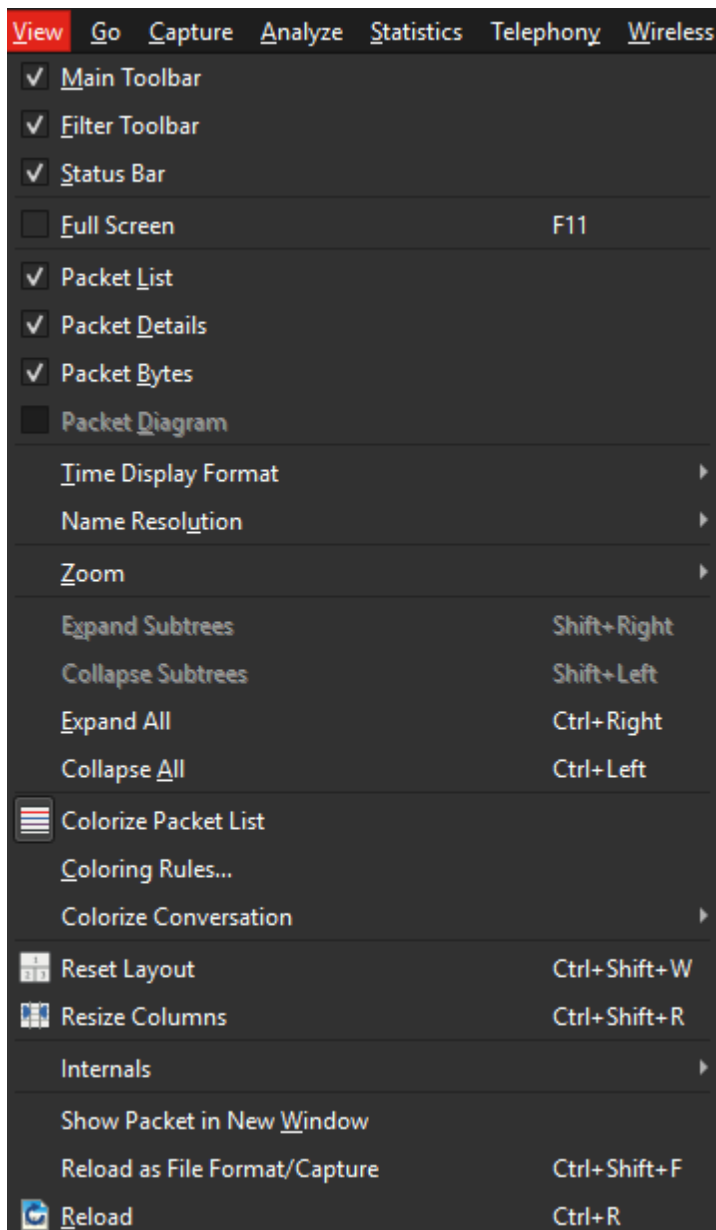
EDIT MENU

Find packets, mark packets for later, and access **Preferences** to change how Wireshark looks.



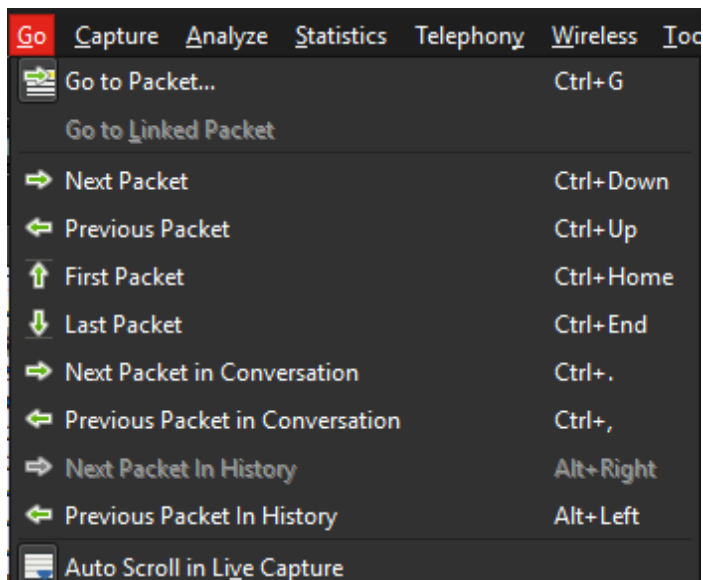
VIEW MENU

Controls what is visible (Toolbars, Status Bar) and how packets are displayed (Time formats).



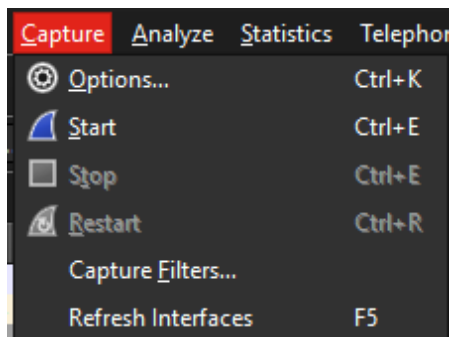
GO MENU

Navigation tools to move between specific packets or "Follow" a stream.



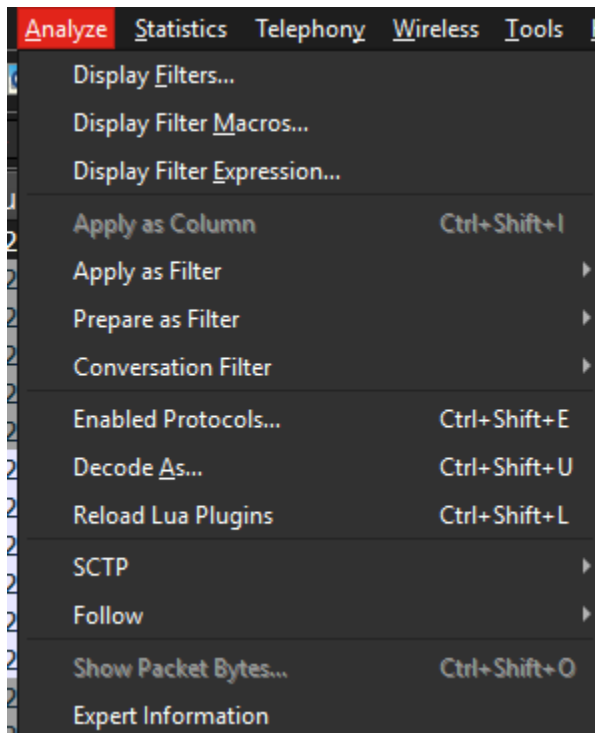
CAPTURE MENU

Settings for live data gathering and selecting network interfaces.



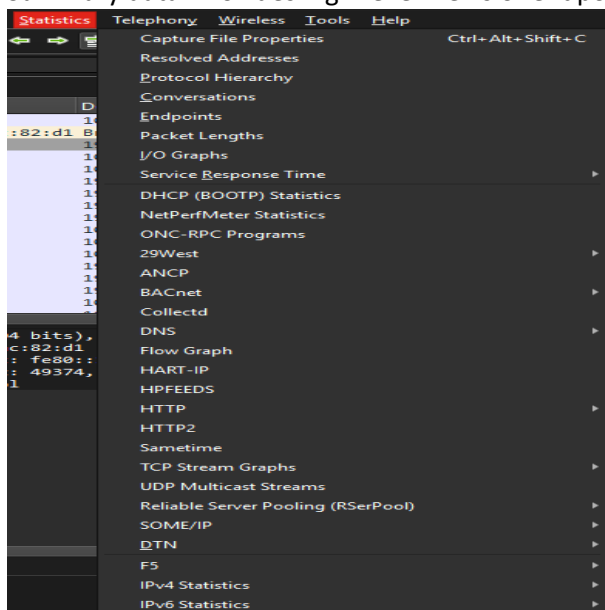
ANALYZE MENU

Where the "logic" happens. Used to enable/disable protocols and follow TCP/HTTP streams



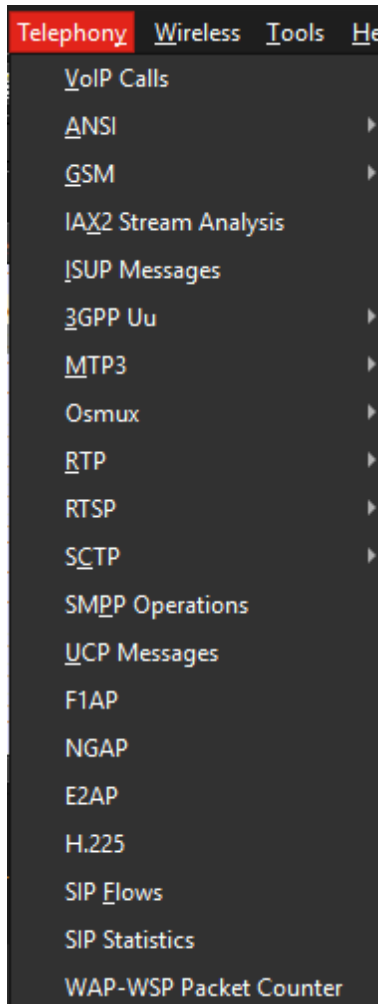
STATISTICS MENU

Summary data. Provides high-level views of endpoints, protocols, and conversation lengths.



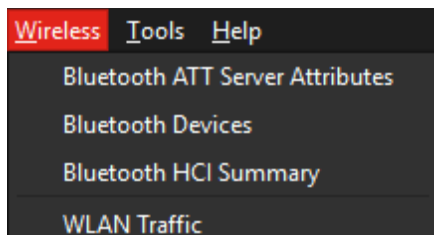
TELEPHONY MENU

Specialized tools for analyzing Voice over IP (VoIP) and cellular traffic.



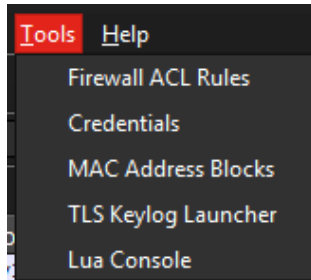
WIRELESS MENU

Tools for Bluetooth and Wi-Fi-specific packet analysis.



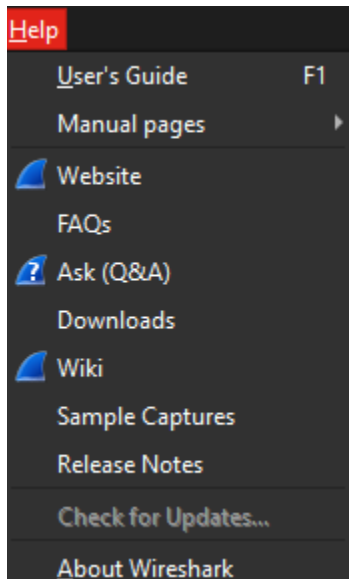
TOOLS MENU

External helpers like Lua scripting or firewall ACL rules.

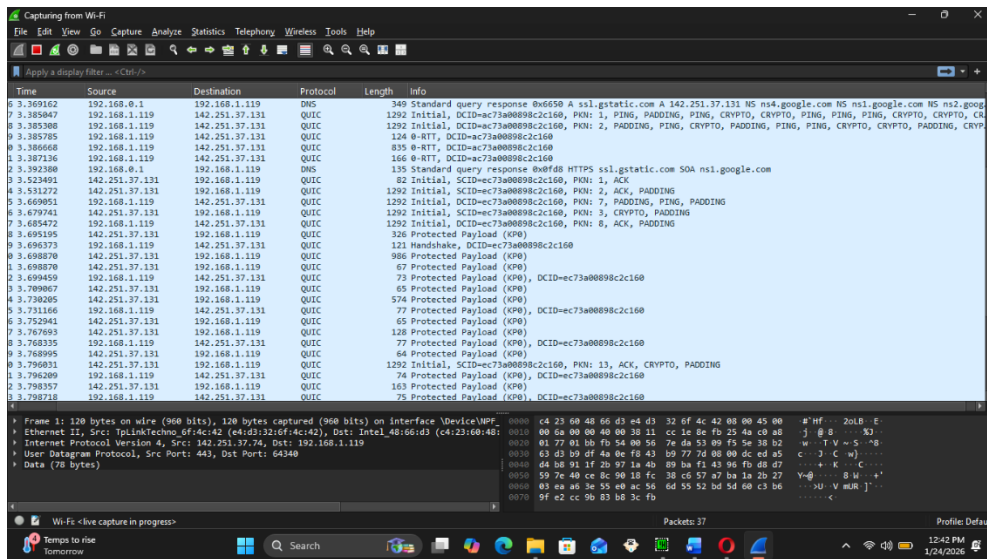


HELP MENU

Access to the manual, Wiki, and "About" information.



MAIN SCREEN

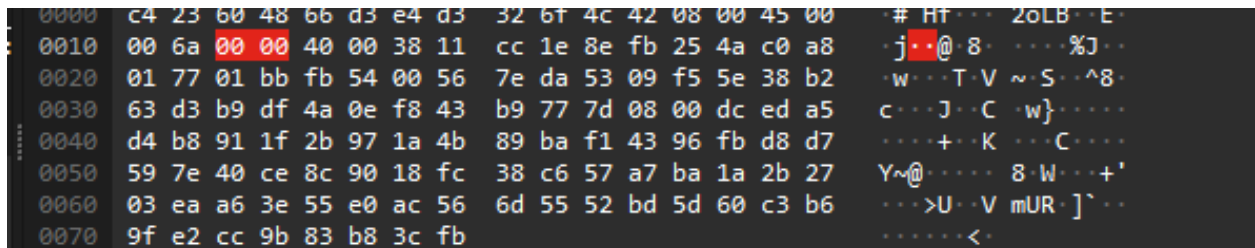


SEARCH FILTER

The bar where you type commands (e.g., tcp) to hide unwanted traffic.

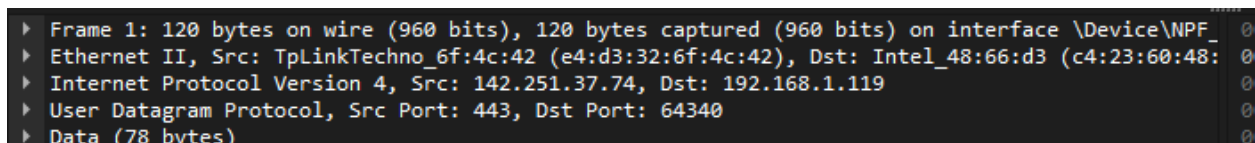
BIT INFORMARTION

The bottom pane showing the raw data in Hexadecimal and ASCII.



FRAMES AND INFORMATIONS

The middle pane that breaks down the packet by layer (Ethernet, IP, TCP, etc.).



```

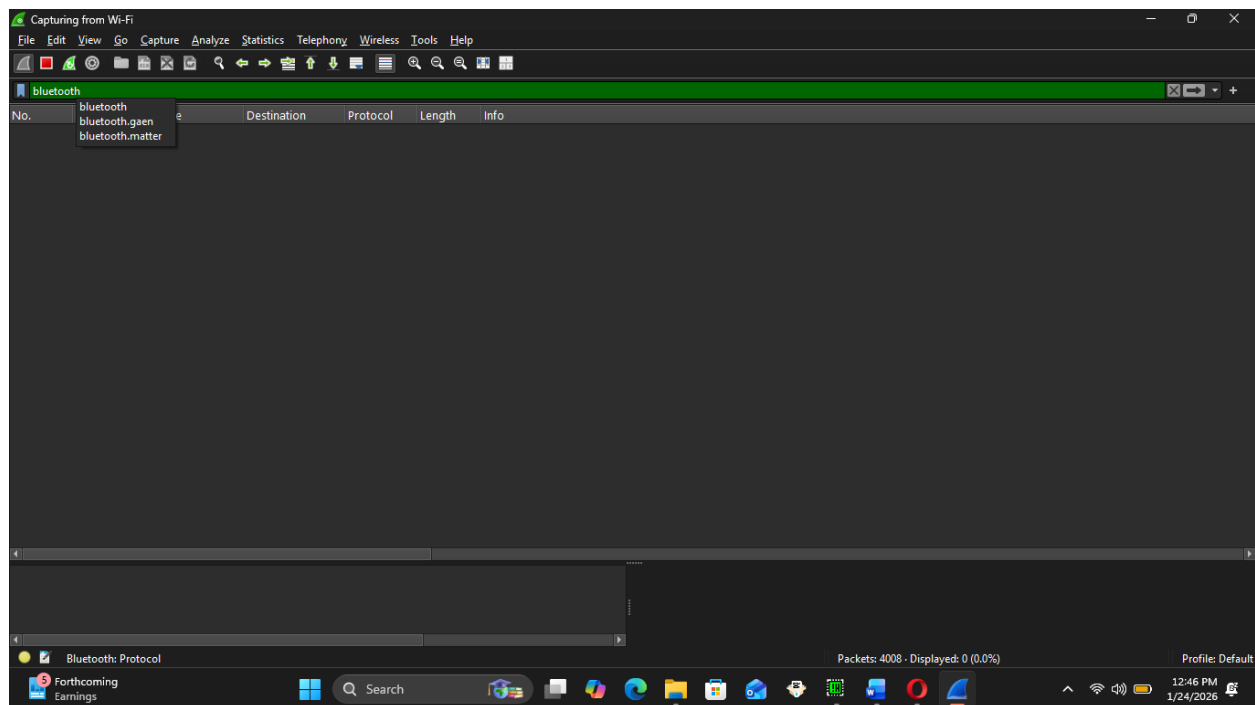
  ▸ Interface id: 0 (\Device\NPF_{B92DCC02-0668-477C-8AD3-B441ECD08B67})
  Encapsulation type: Ethernet (1)
  Arrival Time: Jan 24, 2026 12:42:13.788854000 Pakistan Standard Time
  UTC Arrival Time: Jan 24, 2026 07:42:13.788854000 UTC
  Epoch Arrival Time: 1769240533.788854000
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 0.000000000 seconds]
  [Time delta from previous displayed frame: 0.000000000 seconds]
  [Time since reference or first frame: 0.000000000 seconds]
  Frame Number: 1
  Frame Length: 120 bytes (960 bits)
  Capture Length: 120 bytes (960 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:udp:data]
  [Coloring Rule Name: UDP]
  [Coloring Rule String: udp]
  ▾ Ethernet II, Src: TpLinkTechno_6f:4c:42 (e4:d3:32:6f:4c:42), Dst: Intel_48:66:d3 (c4:23:60:48:66:d3)
    ▸ Destination: Intel_48:66:d3 (c4:23:60:48:66:d3)
    ▸ Source: TpLinkTechno_6f:4c:42 (e4:d3:32:6f:4c:42)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  ▾ Internet Protocol Version 4, Src: 142.251.37.74, Dst: 192.168.1.119
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 106
    Identification: 0x0000 (0)
    ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 56
    Protocol: UDP (17)
    Header Checksum: 0xcc1e [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 142.251.37.74
    Destination Address: 192.168.1.119
    [Stream index: 0]

```

FILTERS:

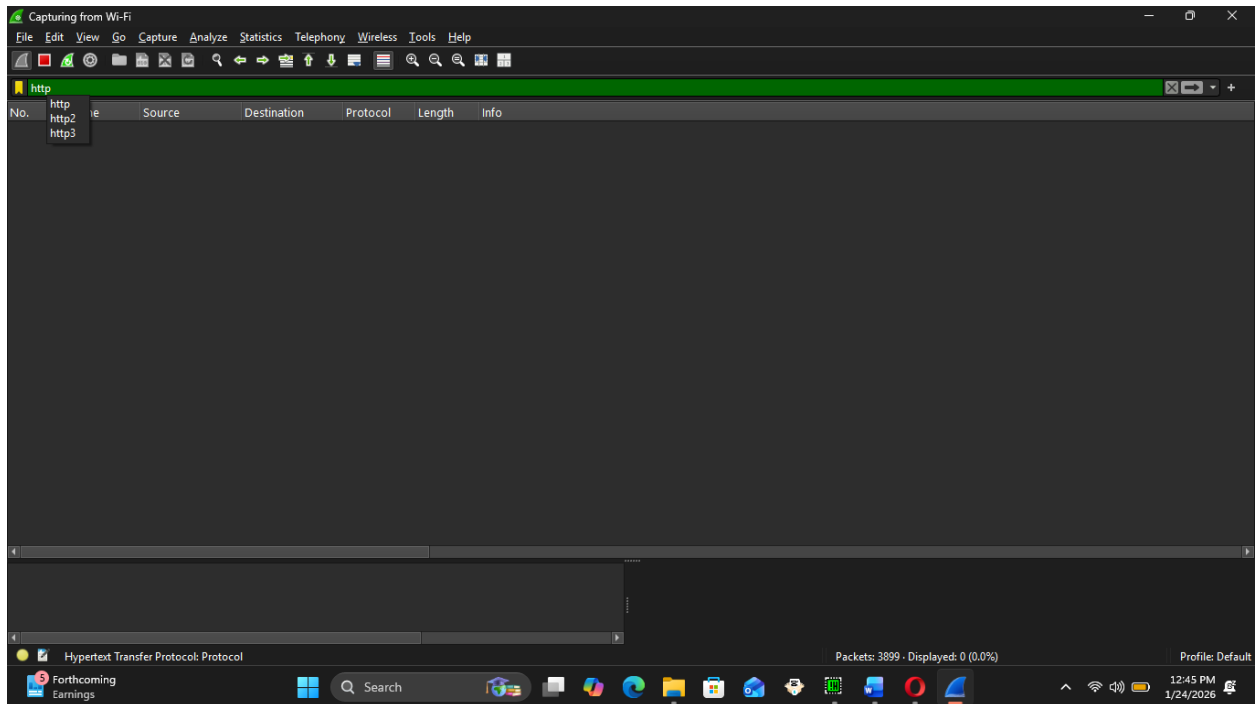
BLUETOOTH

Filters for local wireless device communication.



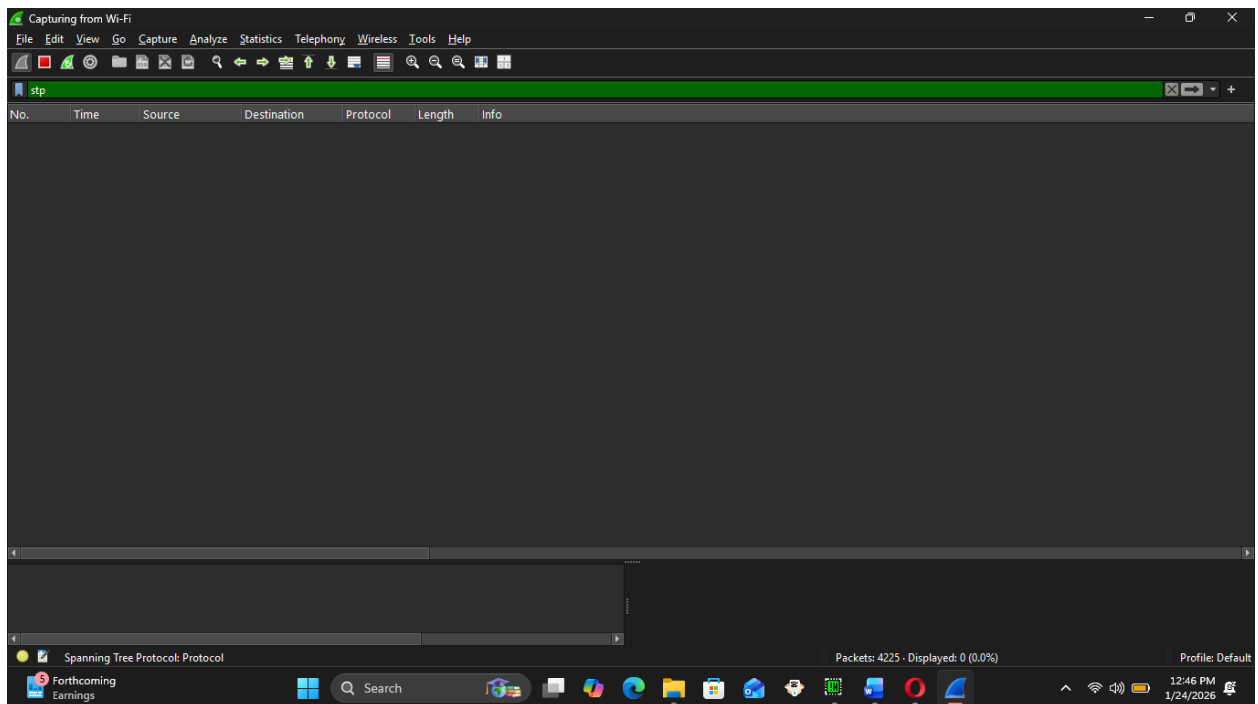
HTTP

shows unencrypted web traffic (great for seeing website data).



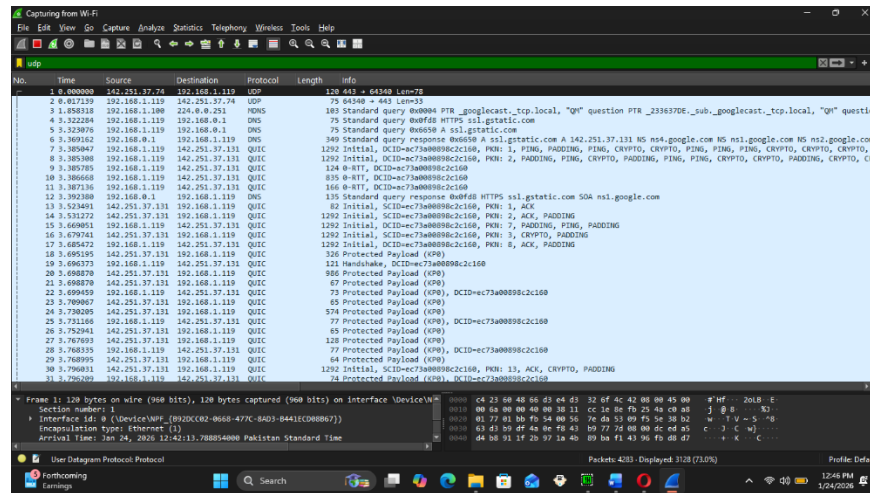
STP

Shows Spanning Tree Protocol, used for network loop prevention.



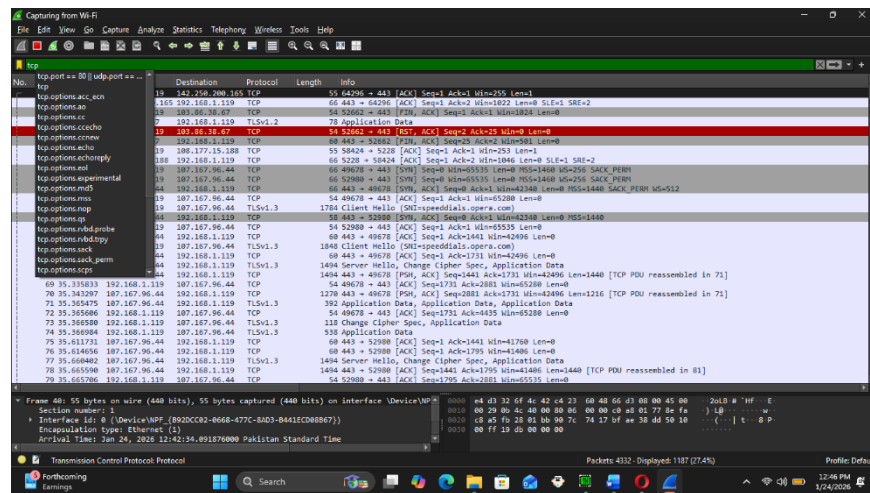
UDP

Shows "Best Effort" traffic like DNS or streaming.



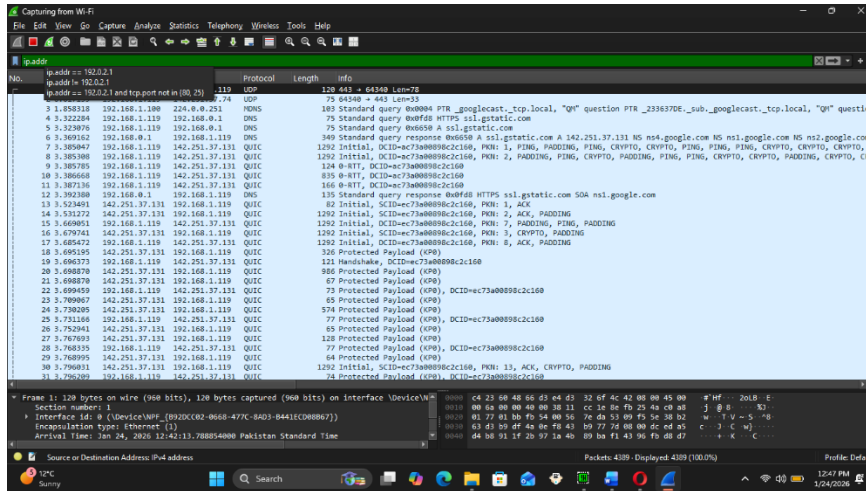
TCP

Shows "Reliable" traffic like file transfers or emails.

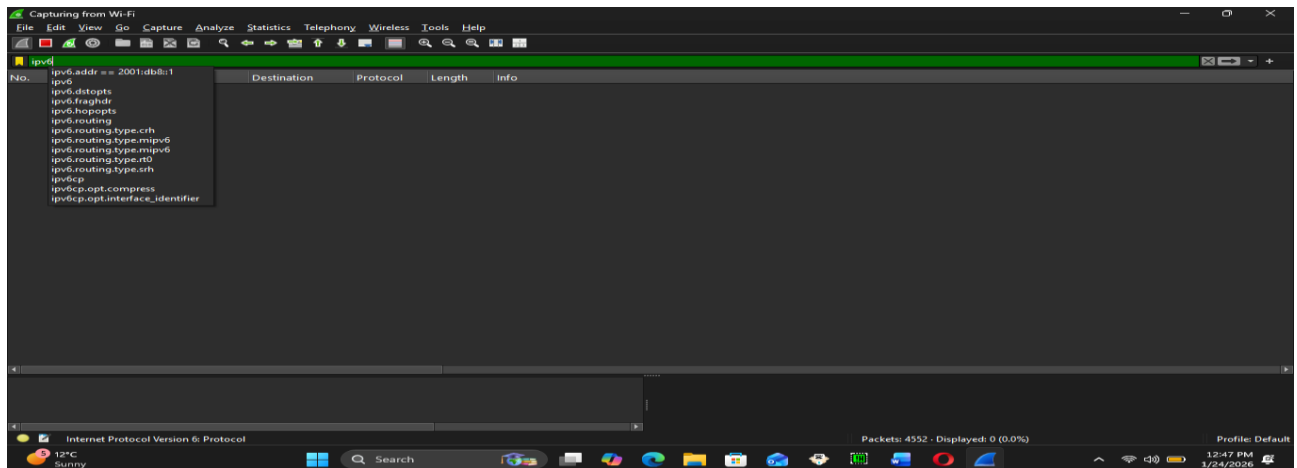


IP

IPV4



IPV6

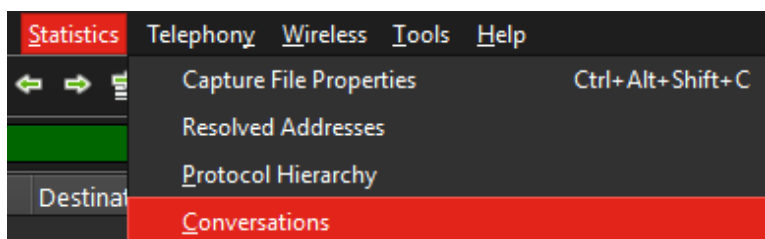


Filters based on the internet protocol version or specific IP addresses.

CONVERSATIONS:

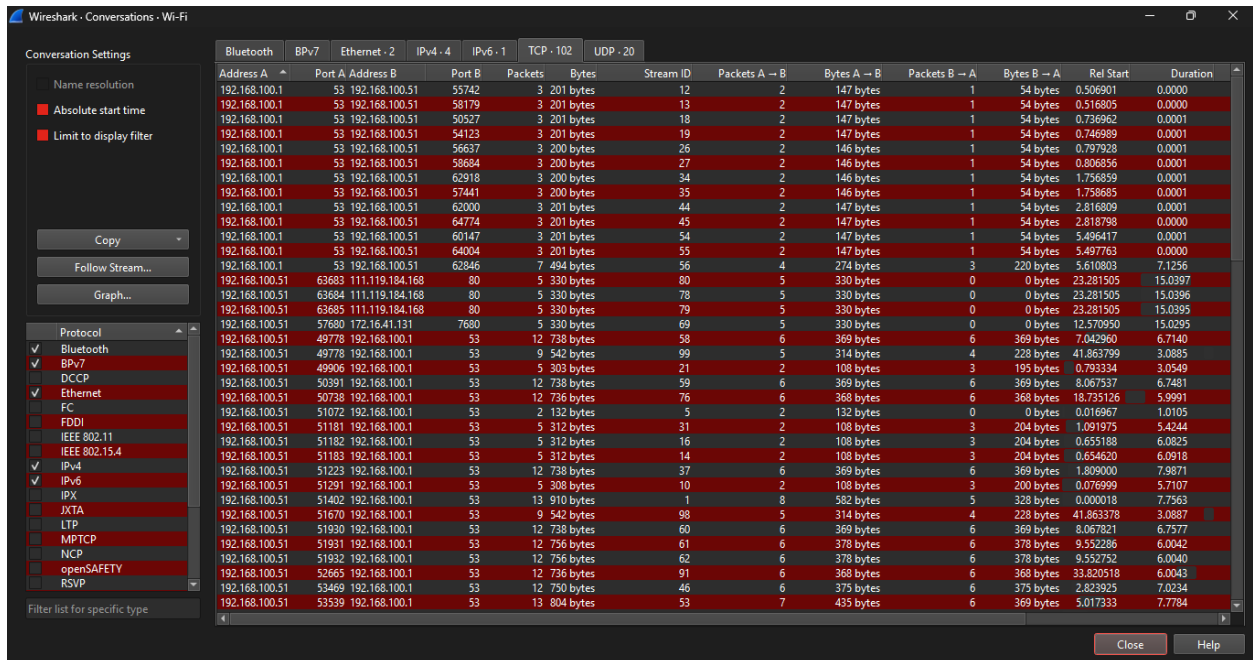
Location

Found under Statistics -> Conversations.



Conversations Screen

A window showing "who is talking to whom" in pairs.

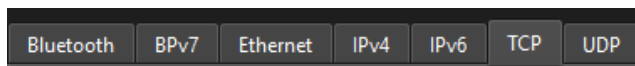


The screenshot shows the 'Conversations' window in Wireshark, filtered for 'Wi-Fi'. The window displays a list of conversations between various IP addresses and ports. The columns include Address A, Port A, Address B, Port B, Packets, Bytes, Stream ID, Packets A → B, Bytes A → B, Packets B → A, Bytes B → A, Rel Start, and Duration. The list is sorted by Ethernet, IPv4, IPv6, TCP, and UDP. The 'Protocol' column on the left shows the protocol for each conversation, with 'Ethernet' and 'IPv4' selected.

Protocol	Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration
Ethernet	192.168.100.1	53	192.168.100.51	55742	3	201 bytes	12	2	147 bytes	1	54 bytes	0.506901	0.0000
	192.168.100.1	53	192.168.100.51	58179	3	201 bytes	13	2	147 bytes	1	54 bytes	0.516805	0.0000
	192.168.100.1	53	192.168.100.51	50527	3	201 bytes	18	2	147 bytes	1	54 bytes	0.736962	0.0001
	192.168.100.1	53	192.168.100.51	54123	3	201 bytes	19	2	147 bytes	1	54 bytes	0.746989	0.0001
	192.168.100.1	53	192.168.100.51	56637	3	200 bytes	26	2	146 bytes	1	54 bytes	0.797928	0.0001
	192.168.100.1	53	192.168.100.51	58684	3	200 bytes	27	2	146 bytes	1	54 bytes	0.806856	0.0001
	192.168.100.1	53	192.168.100.51	62918	3	200 bytes	34	2	146 bytes	1	54 bytes	1.756859	0.0001
	192.168.100.1	53	192.168.100.51	57441	3	200 bytes	35	2	146 bytes	1	54 bytes	1.758885	0.0001
	192.168.100.1	53	192.168.100.51	62000	3	201 bytes	44	2	147 bytes	1	54 bytes	2.816809	0.0001
	192.168.100.1	53	192.168.100.51	64774	3	201 bytes	45	2	147 bytes	1	54 bytes	2.818798	0.0000
IPv4	192.168.100.1	53	192.168.100.51	60147	3	201 bytes	54	2	147 bytes	1	54 bytes	5.486417	0.0001
	192.168.100.1	53	192.168.100.51	64004	3	201 bytes	55	2	147 bytes	1	54 bytes	5.487763	0.0000
	192.168.100.1	53	192.168.100.51	62846	7	494 bytes	56	4	274 bytes	3	220 bytes	5.610803	7.1256
	192.168.100.51	63683	111.119.184.168	80	5	330 bytes	80	5	330 bytes	0	0 bytes	23.281505	15.0397
	192.168.100.51	63684	111.119.184.168	80	5	330 bytes	78	5	330 bytes	0	0 bytes	23.281505	15.0396
	192.168.100.51	63685	111.119.184.168	80	5	330 bytes	79	5	330 bytes	0	0 bytes	23.281505	15.0395
	192.168.100.51	57680	172.16.41.131	7680	5	330 bytes	69	5	330 bytes	0	0 bytes	12.570950	15.0295
	192.168.100.51	49778	192.168.100.1	53	12	738 bytes	58	6	369 bytes	6	369 bytes	7.042960	6.7140
	192.168.100.51	49778	192.168.100.1	53	9	542 bytes	99	5	314 bytes	4	228 bytes	41.863799	3.0885
	192.168.100.51	49906	192.168.100.1	53	5	303 bytes	21	2	108 bytes	3	195 bytes	0.793334	3.0549

Types of Conversations

Sorted by Ethernet, IPv4, IPv6, TCP, or UDP.

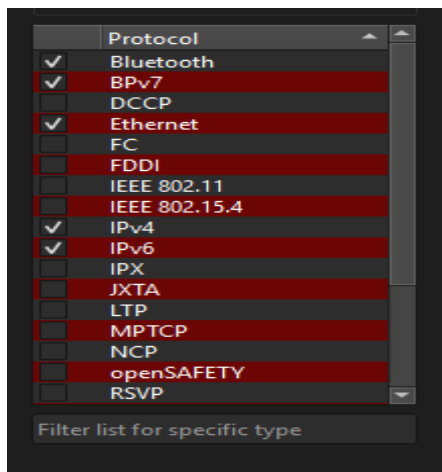


The screenshot shows the 'Conversation Types' window in Wireshark. It displays a list of conversation types: Bluetooth, BPv7, Ethernet, IPv4, IPv6, TCP, and UDP. The 'Ethernet' and 'IPv4' types are selected.

Bluetooth	BPv7	Ethernet	IPv4	IPv6	TCP	UDP
☐	☐	☒	☒	☐	☐	☐

Protocols filter

Limit the list to only show specific types of traffic.



The screenshot shows the 'Filter' window in Wireshark. It displays a list of protocols: Bluetooth, BPv7, DCCP, Ethernet, FC, FDDI, IEEE 802.11, IEEE 802.15.4, IPv4, IPv6, IPX, JXTA, LTP, MPTCP, NCP, openSAFETY, and RSVP. The 'Ethernet' and 'IPv4' protocols are selected.

Protocol
☒ Bluetooth
☒ BPv7
☐ DCCP
☒ Ethernet
☐ FC
☐ FDDI
☐ IEEE 802.11
☐ IEEE 802.15.4
☒ IPv4
☒ IPv6
☐ IPX
☐ JXTA
☐ LTP
☐ MPTCP
☐ NCP
☐ openSAFETY
☐ RSVP

Conversation properties

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel St
-----------	--------	-----------	--------	---------	-------	-----------	---------------	------------------	---------------	-------------	---------------	-------------	--------

Address A/B: The two devices involved.

Port A/B: The "doors" used for the chat (e.g., Port 80 for Web).

Packets/Bytes: Total amount of data shared.

Packets A->B / B->A: Who is doing most of the talking?

Rel Start / Duration: When the chat started and how long it lasted.

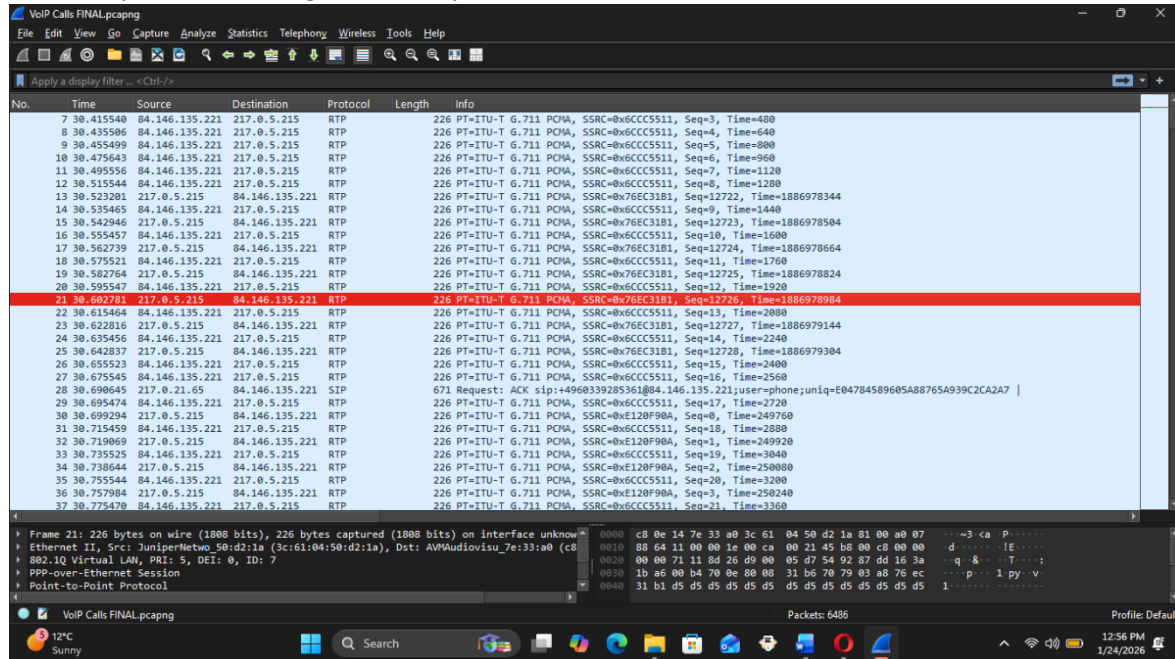
Applying Filter: Right-click any conversation to "Apply as Filter" and see only that specific chat on the main screen.

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration
192.168.100.1		Apply as Filter		Selected		Filter on stream id	2	147 bytes	1	54 bytes	0.506901	0.0000
192.168.100.1		Prepare as Filter		Not Selected		A → B	2	147 bytes	1	54 bytes	0.516805	0.0000
192.168.100.1		Find		...and Selected		A → B	2	147 bytes	1	54 bytes	0.746989	0.0001
192.168.100.1		Colorize		...or Selected		B → A	2	146 bytes	1	54 bytes	0.797928	0.0001
192.168.100.1		Copy Conversation table		...and not Selected		A → Any	2	146 bytes	1	54 bytes	0.806856	0.0001
192.168.100.1		Resize all columns to content		...or not Selected		A → Any	2	146 bytes	1	54 bytes	1.758659	0.0001
192.168.100.1						A → Any	2	147 bytes	1	54 bytes	2.816809	0.0001
192.168.100.1	53	192.168.100.51	64774	3	201 bytes	Any → A	2	147 bytes	1	54 bytes	2.818798	0.0000
192.168.100.1	53	192.168.100.51	60147	3	201 bytes	Any → B	2	147 bytes	1	54 bytes	5.496417	0.0001
192.168.100.1	53	192.168.100.51	64004	3	201 bytes	Any → B	2	147 bytes	1	54 bytes	5.497763	0.0000
192.168.100.1	53	192.168.100.51	62846	7	494 bytes	Any → B	4	274 bytes	3	220 bytes	5.610803	7.1256
192.168.100.51	63683	111.119.184.168	80	5	330 bytes	B → Any	5	330 bytes	0	0 bytes	23.281505	15.0397
192.168.100.51	63684	111.119.184.168	80	5	330 bytes	B → Any	5	330 bytes	0	0 bytes	23.281505	15.0396

VOIP CALLS

DOWNLOAD AND IMPORT TO WIRESHARK

Load a capture containing SIP or RTP packets.

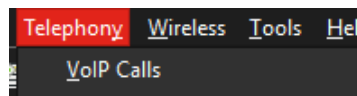


PLAY STREAMS

Hear the actual audio of the captured phone call.

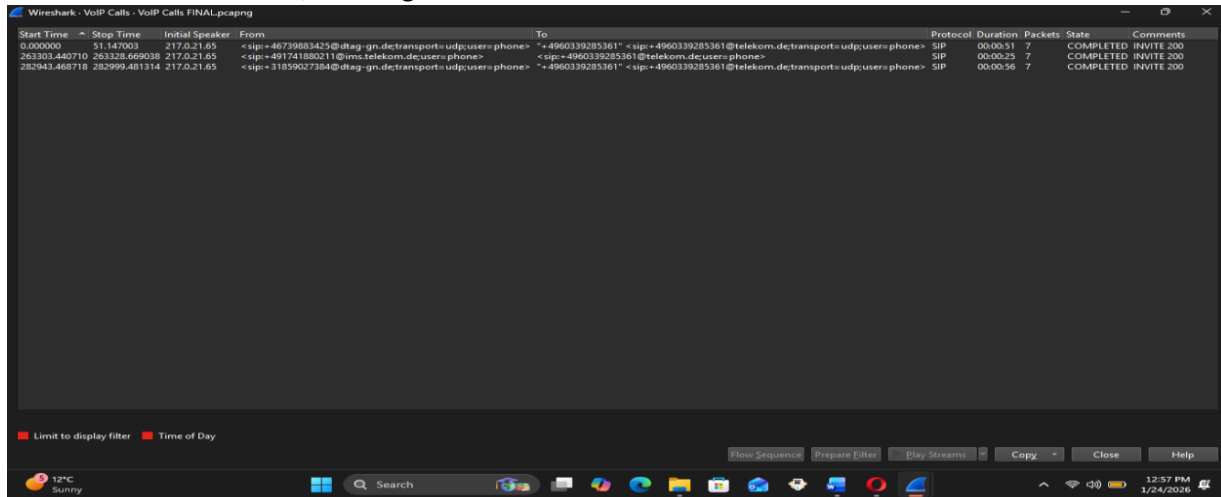
LOCATION:

Telephony -> VoIP Calls.



CALL RECORDS

A list of all detected calls, showing start time and status.

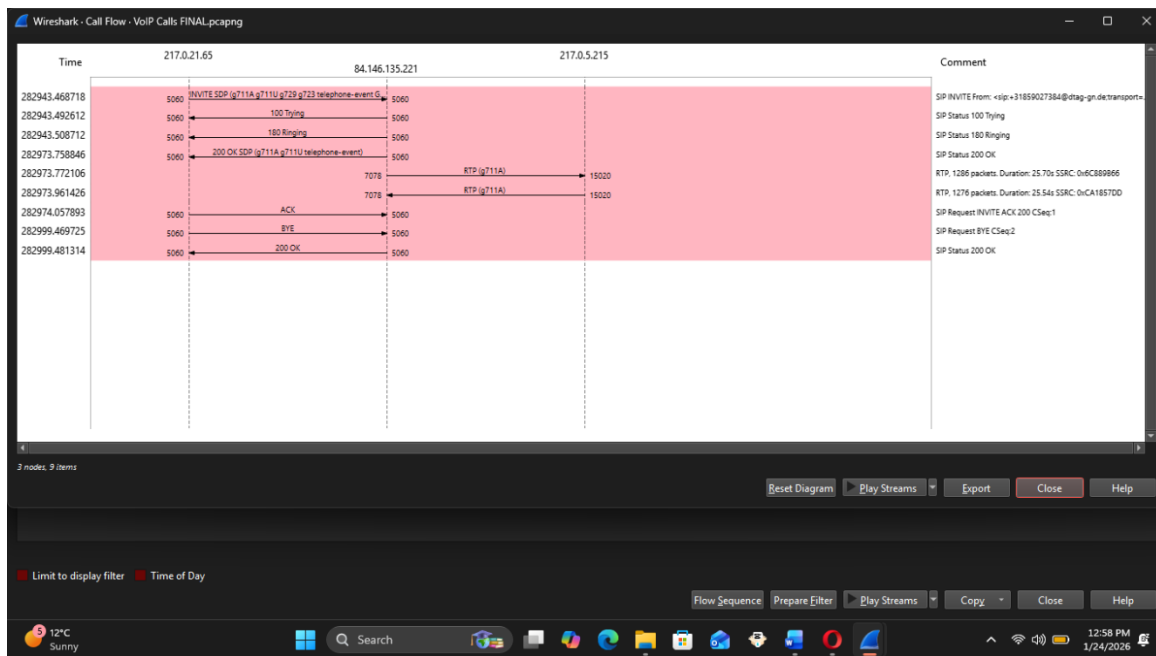


The screenshot shows the Wireshark interface with a packet list for 'VoIP Calls - VoIP Calls FINAL.pcapng'. The list contains three SIP messages: an INVITE, a 200 OK, and another INVITE. The interface includes a packet list pane, a packet details pane, and a packet bytes pane. The status bar at the bottom shows the system time as 12:57 PM on 1/24/2026.

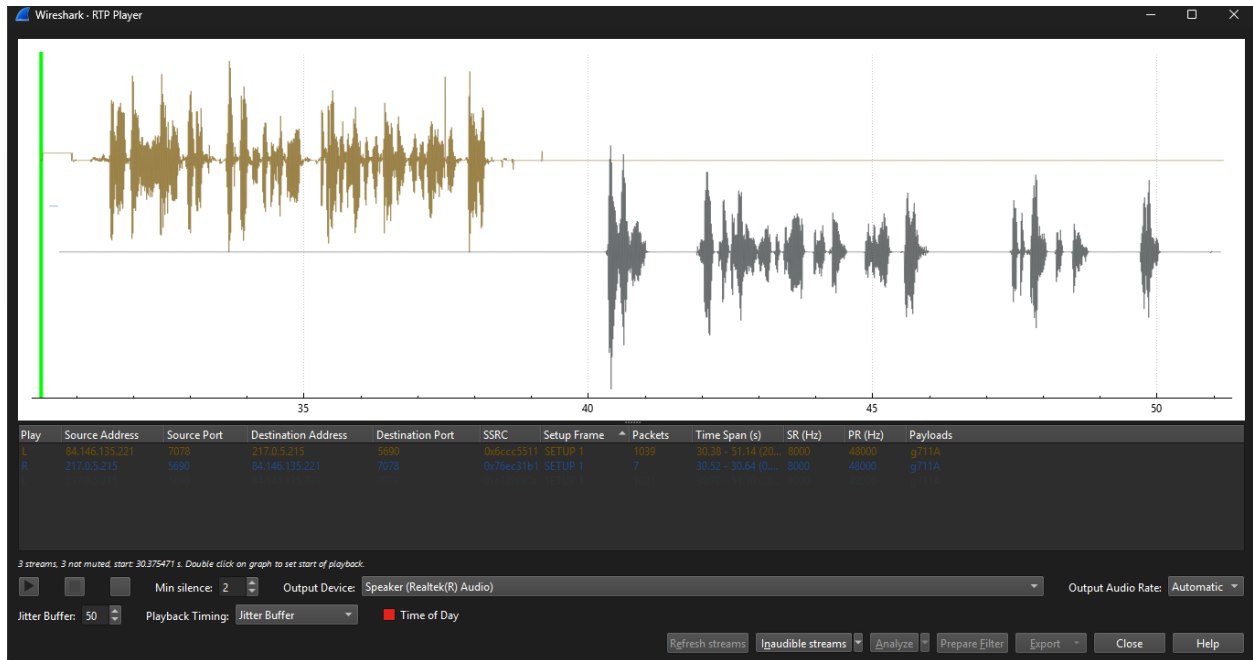
Start Time	Stop Time	Initial Speaker	From	To	Protocol	Duration	Packets	State	Comments
0.000000	51.147003	217.0.21.65	< sip+46739883425@dtag-gn.de;transport=udp;user=phone>	<+4960339285361" < sip+4960339285361@telekom.de;transport=udp;user=phone>	SIP	00:00:51	7	COMPLETED	INVITE 200
263303.440710	263328.669038	217.0.21.65	< sip+491741880211@ims.telekom.de;user=phone>	< sip+4960339285361@telekom.de;user=phone>	SIP	00:00:25	7	COMPLETED	INVITE 200
282943.468718	282999.481314	217.0.21.65	< sip+31859027384@dtag-gn.de;transport=udp;user=phone>	<+4960339285361" < sip+4960339285361@telekom.de;transport=udp;user=phone>	SIP	00:00:56	7	COMPLETED	INVITE 200

FLOW SEQUENCE

A visual "ladder diagram" showing how the call was connected and disconnected.



PLAY STREAMS:



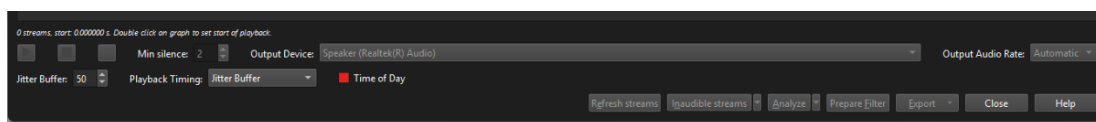
CALL INTERNET INFORMATION

Details on the jitter and packet loss of the cal

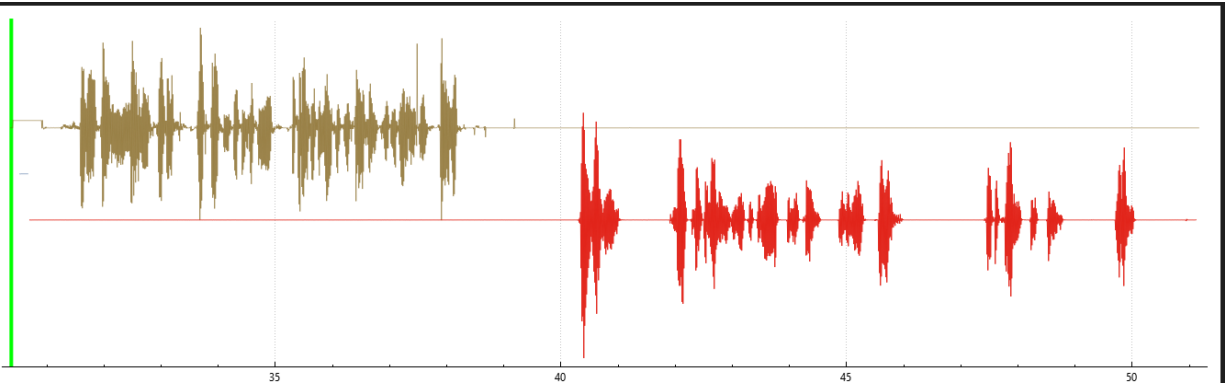
Play	Source Address	Source Port	Destination Address	Destination Port	SSRC	Setup Frame	Packets	Time Span (s)	SR (Hz)	PR (Hz)	Payloads
L	84.146.135.221	7078	217.0.5.215	5690	0x6ccc5511	SETUP 1	1039	30.38 - 51.14 (20...	8000	48000	g711A
R	217.0.5.215	5690	84.146.135.221	7078	0x76ec3...	SETUP 1	7	30.52 - 30.64 (...	8000	48000	g711A

CALL CONTROLS

Tools to analyze the quality and timing of the voice data.



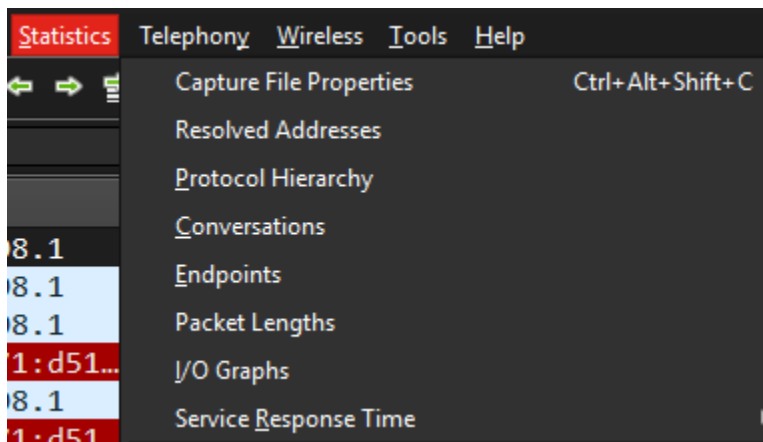
CALL GRAPHS



I/O GRAPHS:

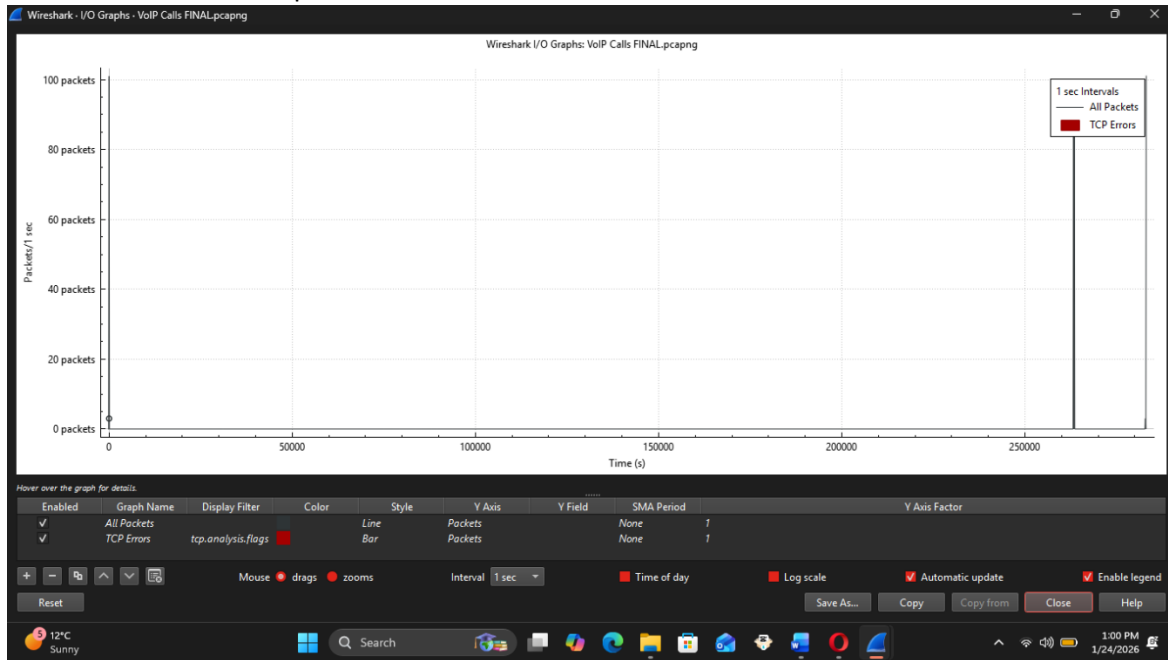
LOCATION

Statistics -> I/O Graphs



GRAPHS SCREEN:

A visual chart of traffic spikes and lulls.



GRAPH INFORMATION

Shows data like "Packets per Second" or "Bits per Second."

No packets in interval (86779s)

Enabled	Graph Name	Display Filter	Color	Style	Y Axis	Y Field	SMA Period	Y Axis Factor
✓	All Packets			Line	Packets	Packets	None	1
✓	TCP Errors	tcp.analysis.flags		Bar	Packets	Packets	None	1

GRAPH CONTROLS

Allows you to change colors, styles (Line vs. Bar), and time intervals to spot anomalies like a DDoS attack.



Encryption and Decryption

Caesar Cipher (Shift Cipher):

Explanation:

The Caesar Cipher is one of the oldest and simplest forms of encryption. Named after Julius Caesar, who used it to protect his military secrets, it is a **Substitution Cipher**. This means each letter in the original message is replaced by a letter a fixed number of positions down the alphabet.

How it works

The Key: A number (from 1 to 25) that represents the "Shift."

The Algorithm: Move each letter of your message forward by the value of the Key.

If you reach 'Z', you "wrap around" back to 'A'.

Code:

```
def caesar_cipher(text, shift, mode='encrypt'):
    result = ""

    # If decrypting, we simply reverse the shift
    if mode == 'decrypt':
        shift = -shift

    for char in text:
        if char.isalpha(): # Check if the character is a letter
            start = ord('A') if char.isupper() else ord('a')
            # The Formula: (Position + Shift) % 26
            new_char = chr(start + (ord(char) - start + shift) % 26)
            result += new_char
        else:
            # Keep spaces and punctuation as they are
            result += char

    return result

# --- Lab Test ---
message = "Information Security 2026"
```

```
key = 5

# Encrypting
secret = caesar_cipher(message, key, mode='encrypt')
print(f"Ciphertext: {secret}")

# Decrypting
original = caesar_cipher(secret, key, mode='decrypt')
print(f"Plaintext: {original}")
```

Symmetric Encryption (Secret Key)

The Concept:

The same key is used to both lock (encrypt) and unlock (decrypt) the data. It is extremely fast and ideal for encrypting large amounts of data (like a hard drive).

Real-world Example: AES (Advanced Encryption Standard), used by your Wi-Fi (WPA2/WPA3).

The Code (Python - using cryptography library)

```
from cryptography.fernet import Fernet

# 1. Generate a Key
key = Fernet.generate_key()
cipher_suite = Fernet(key)

# 2. Encrypt
message = b"Top Secret Lab Data"
cipher_text = cipher_suite.encrypt(message)
print(f"Encrypted: {cipher_text}")

# 3. Decrypt
plain_text = cipher_suite.decrypt(cipher_text)
print(f"Decrypted: {plain_text.decode()}")
```

Asymmetric Encryption (Public Key)

The Concept:

Uses a "Key Pair." A **Public Key** (can be shared with anyone) and a **Private Key** (must be kept secret). If you encrypt with the Public Key, only the owner of the Private Key can decrypt it.

Real-world Example: RSA, used for digital signatures and secure web connections (HTTPS).

The Code (Python - RSA)

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# 1. Generate Key Pair
private_key = rsa.generate_private_key(public_exponent=65537,
key_size=2048)
public_key = private_key.public_key()

# 2. Encrypt using Public Key
message = b"Asymmetric Secret"
cipher_text = public_key.encrypt(
    message,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algo-
rithm=hashes.SHA256(), label=None)
)

# 3. Decrypt using Private Key
plain_text = private_key.decrypt(
    cipher_text,
    padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algo-
rithm=hashes.SHA256(), label=None)
)
print(f"Decrypted: {plain_text.decode()}")
```

Monoalphabetic Cipher

What it is

Unlike the Caesar Cipher (which shifts the whole alphabet by a fixed number), a **Monoalphabetic Cipher** maps each letter of the alphabet to a completely random substitute letter.

The Key: A "Substitution Map" or a scrambled alphabet.

The Strength: There are $26!$ (26 factorial) possible keys, which is a massive number (4×10^{26}), making it much harder to brute-force than the Caesar Cipher.

The Code (Python)

```
import random

def monoalphabetic_encrypt(text, key_map):
    return "".join(key_map.get(c, c) for c in text.upper())

# Creating a random key map
alphabet = list("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
shuffled = list(alphabet)
random.shuffle(shuffled)
key_map = dict(zip(alphabet, shuffled))

message = "HELLO WORLD"
encrypted = monoalphabetic_encrypt(message, key_map)
print(f"Key Map: {key_map}")
print(f"Encrypted: {encrypted}")
```

One-Time Pad (OTP)

What it is

The One-Time Pad is the **only mathematically unbreakable** encryption. It uses a pre-shared key (the "pad") that is at least as long as the message itself.

Rule: The key must be truly random, used only once, and kept completely secret.

How it works: Each bit or character of the plaintext is combined with a bit or character from the pad (usually using XOR logic).

The Code (Python)

```
def xor_cipher(text, key):
    # XOR each character with the corresponding key character
    return "".join(chr(ord(t) ^ ord(k)) for t, k in zip(text, key))

message = "SECRET"
key = "XMCKLH" # Key must be same length as message

encrypted = xor_cipher(message, key)
decrypted = xor_cipher(encrypted, key) # XORing again with same key de-
crypts it

print(f"Ciphertext (raw): {repr(encrypted)}")
print(f"Decrypted: {decrypted}")
```

Transposition Cipher

What it is

Transposition does **not** change the letters (like substitution). Instead, it **rearranges the order** of the letters. It's like a professional-grade anagram.

Rail Fence Cipher: A common type where you write the message in a zigzag pattern across "rails" and then read it off row by row.

Columnar Transposition: Writing the message in rows of a fixed length and reading it back column by column based on a keyword.

The Code (Rail Fence Example)

```
def rail_fence_encrypt(text, rails):
    fence = [[] for _ in range(rails)]
    rail = 0
    direction = 1

    for char in text:
        fence[rail].append(char)
        rail += direction
        if rail == 0 or rail == rails - 1:
            direction *= -1

    return "".join(["".join(r) for r in fence])

message = "TRANSPPOSITION"
print(f"Encrypted (2 rails): {rail_fence_encrypt(message, 2)}")
```